SPARK PERFORMANCE TUNING

A Practical Checklist for Diagnosing Slow Jobs

Spark UI | Partitions | Joins | AQE

Interview Ready in 6 Slides

Swipe to start learning →

U **START WITH THE SPARK UI**

Every tuning decision should start from evidence in the Spark UI — guessing wastes time and can make things worse.

Where to Look First

1. **Stages tab** — find the longest-running Stage and its shuffle read/write size

↓

2. **Tasks summary** — check min/median/max duration; a big gap signals skew

↓

3. **SQL tab** — inspect the physical plan for unexpected shuffles or missed broadcasts

↓

4. **Executors tab** — check GC time and spill for memory pressure

Interview Tip: If you can't point to a specific Stage or metric, you're not ready to change a config yet.

L **THE HIGH-LEVERAGE CHANGES**

WHERE MOST WINS COME FROM

Partitioning | Caching | Broadcast Joins | File Format | AQE

What to Actually Change

1. Right-size **spark.sql.shuffle.partitions** — too few starves parallelism, too many adds overhead

↓

2. Cache only DataFrames reused across multiple Actions, then **.unpersist()** when done

↓

3. Force **broadcast joins** for small dimension tables Catalyst underestimates

↓

4. Use columnar formats (**Parquet/ORC**) with predicate pushdown over CSV/JSON

Interview Tip: These five levers fix the majority of real-world slow jobs — most other tuning is diminishing returns.

Adaptive Query Execution

A LET SPARK RE-PLAN AT RUNTIME

AQE re-optimizes the physical plan **mid-Job** using actual shuffle statistics instead of only upfront estimates.

What AQE Fixes Automatically

1. Coalesces small post-shuffle partitions — no manual shuffle.partitions tuning

↓

2. Switches Sort-Merge → **Broadcast Join** if runtime stats show one side is small

↓

3. Splits skewed partitions into smaller sub-tasks automatically

↓

4. Enabled by default since Spark 3.2 — `spark.sql.adaptive.enabled`

Interview Tip: AQE turns compile-time guesses into runtime facts — mention it whenever discussing join or partition tuning.

X **FREQUENT MISTAKES TO AVOID**

Most performance regressions trace back to one of these four habits.

What Quietly Kills Performance

! **.collect()** on large DataFrames — pulls everything to the Driver, risking OOM

! **Python UDFs** instead of built-in functions — breaks Whole-Stage CodeGen

! Excessive **.cache()** calls that evict useful blocks and fragment Storage Memory

! Reading **many small files** — creates thousands of tiny partitions and Task overhead

Interview Tip: Prefer built-in Spark SQL functions over UDFs whenever possible — UDFs are a black box to Catalyst.

Interview Cheat Sheet

Save this slide before your next Data Engineering interview

LEVER	FIXES	KEY INTERVIEW FACT
Spark UI	Diagnosis	Always confirm the bottleneck Stage first
Partitioning	Parallelism / skew	Right-size shuffle.partitions
Broadcast Join	Shuffle elimination	Force via broadcast(df) hint
AQE	Runtime re-planning	Enabled by default since Spark 3.2
Avoid UDFs/collect()	CodeGen & OOM	Prefer built-ins, avoid pulling data to Driver

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep